

Learning PDE Operators under Partial Observations: Towards Data-Efficient Scientific Modeling

Hanoi, March 2026

Abstract

1 Introduction

2 Related Work

Neural operators for scientific machine learning. Neural operators have emerged as powerful data-driven surrogates for solving parametric partial differential equations. The Fourier Neural Operator (FNO) [1] learns mappings between function spaces by parameterizing integral kernels in the Fourier domain, achieving strong performance on a variety of PDE benchmarks. Concurrently, DeepONet [2] decomposes operator learning into a branch network (encoding the input function) and a trunk network (encoding query coordinates), offering a flexible architecture applicable to diverse operator learning tasks. These methods typically assume full access to the system state, such as complete initial or boundary conditions defined on a dense spatial grid.

Physics-informed learning. Physics-informed neural networks (PINNs) [3] incorporate governing equations as soft constraints via automatic differentiation, reducing the need for labeled simulation data. This paradigm has been extended to operator learning frameworks [4]. However, physics-informed approaches face two key limitations: (1) they require an analytically tractable PDE residual, which is challenging for complex real-world domains with spatially-varying bathymetry, and (2) they still rely on full-state observations as input. In this work, we focus on the complementary challenge of *input reduction*—enabling accurate prediction from boundary-only sensor data—rather than adding physics constraints to the loss function.

Partial observation and data assimilation. In real-world geophysical systems, full-state observations are rarely available. Measurements are typically limited to sparse sensors deployed at fixed locations or boundary-only signals from satellite altimetry. Classical approaches to this challenge include data assimilation techniques such as the Ensemble Kalman Filter [6] and variational methods (4D-Var). However, these methods are computationally expensive and tightly coupled to specific numerical solvers. In this work, we extend operator learning to the partial-observation regime and demonstrate that DeepONet can accurately reconstruct full-field dy-

namics from drastically reduced input information, achieving comparable accuracy to full-state baselines while using less than 1% of the original input dimensionality.

3 Methodology

3.1 Governing Equations

The physical system under study is governed by the two-dimensional shallow water equations (SWE), derived from depth-averaging the Navier–Stokes equations under the hydrostatic pressure assumption:

$$\frac{\partial h}{\partial t} + \frac{\partial(hu)}{\partial x} + \frac{\partial(hv)}{\partial y} = 0, \quad (1)$$

$$\frac{\partial(hu)}{\partial t} + \frac{\partial}{\partial x}(hu^2 + \tfrac{1}{2}gh^2) + \frac{\partial(huv)}{\partial y} = 0, \quad (2)$$

$$\frac{\partial(hv)}{\partial t} + \frac{\partial(huv)}{\partial x} + \frac{\partial}{\partial y}(hv^2 + \tfrac{1}{2}gh^2) = 0, \quad (3)$$

where $h(x, y, t) \geq 0$ is the water depth, (u, v) are the depth-averaged velocity components, and $g = 9.81 \text{ m/s}^2$ is the gravitational acceleration. Equation (1) expresses conservation of mass; equations (2)–(3) express conservation of momentum.

3.2 Operator Learning Task

Traditional numerical methods solve eqs. (1)–(3) by time-stepping from a given initial condition, which is computationally expensive. Our goal is to replace this iterative solve with a learned solution operator $\mathcal{G}$ that directly maps the scenario description to the full spatiotemporal solution in a single forward pass.

Full-state setting (baseline). The operator receives the complete initial condition field:

$$\mathcal{G}_{\text{full}} : u_0(x, y) \mapsto h(x, y, t), \quad \forall (x, y) \in \Omega, t \in (0, T]. \quad (4)$$

Partial-observation setting (proposed). We assume access to K fixed sensor locations $\{(x_k, y_k)\}_{k=1}^K$, where $K \ll N_x \times N_y$, that record the water depth continuously over time:

$$\mathcal{G}_{\text{partial}} : \mathbf{s}(t) = [h(x_1, y_1, t), \dots, h(x_K, y_K, t)]_{t=t_0}^T \mapsto h(x, y, t). \quad (5)$$

The operator never sees the spatial structure of the field at any single instant; it must reconstruct global dynamics solely from the temporal correlations embedded in the sparse sensor signals.

3.3 DeepONet Architecture

We employ the Deep Operator Network (DeepONet) [2], which decomposes the operator into two cooperating subnetworks.

Branch Network $\mathcal{B}$. Encodes the input function (scenario description) into a latent vector:

$$\mathcal{B} : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^p, \quad \mathbf{b} = \mathcal{B}(\mathbf{u}_{\text{input}}), \quad (6)$$

where d_{in} depends on the observation regime and p is the latent dimension.

Trunk Network $\mathcal{T}$. Encodes the query location (where and when to evaluate the solution):

$$\mathcal{T} : (\tilde{x}, \tilde{y}, \tilde{t}) \in [-1, 1]^3 \rightarrow \mathbb{R}^p, \quad \mathbf{t} = \mathcal{T}(\tilde{x}, \tilde{y}, \tilde{t}), \quad (7)$$

where coordinates are normalized via min-max scaling.

Output combination. The prediction at any query point is obtained via inner product plus a learnable bias:

$$\hat{h}(x, y, t) = \sum_{i=1}^p b_i \cdot t_i + b_0. \quad (8)$$

This is analogous to a basis expansion: the trunk learns universal spatial modes, while the branch learns scenario-specific coefficients.

Both networks are implemented as 4-layer MLPs with width 256, GELU activation [7], and latent dimension $p = 128$, totaling $\sim 4.66 \times 10^6$ parameters.

3.4 Partial Observation Encoding

The distinction between settings lies entirely in the branch input.

Full-state input. The branch receives the flattened initial condition:

$$\mathbf{u}_{\text{full}} = \text{flatten}[h(x_i, y_j, t=0)] \in \mathbb{R}^{N_x \cdot N_y}. \quad (9)$$

For PDEBench ($N_x = N_y = 128$): $d_{\text{in}} = 16,384$.

Partial input (proposed). K sensors on a uniform $\sqrt{K} \times \sqrt{K}$ sub-grid record time-series across all N_t steps:

$$\mathbf{u}_{\text{partial}} = \text{flatten}[h(x_k, y_k, t_j)]_{k=1, \dots, K}^{j=1, \dots, N_t} \in \mathbb{R}^{K \cdot N_t}. \quad (10)$$

With $K = 16$ sensors and $N_t = 101$: $d_{\text{in}} = 1,616$, a $\sim 90\%$ reduction.

We further distinguish **interior sensors** (distributed across the domain) from **boundary-only sensors** (placed exclusively on the four edges). The latter directly mimics real-world tidal gauge deployments.

3.5 Training Objective

The model minimizes the mean squared error between predictions and reference numerical solutions at randomly sampled query points:

$$\mathcal{L} = \frac{1}{M} \sum_{m=1}^M (\hat{h}(x_m, y_m, t_m) - h^*(x_m, y_m, t_m))^2, \quad (11)$$

where h^* denotes the reference solution. All models are trained with AdamW [8] ($\alpha_0 = 10^{-3}$, weight decay 10^{-4}), cosine annealing schedule, gradient clipping (max norm 1.0), for 50 epochs.

4 Experiments

4.1 Benchmark on PDEBench 2D Shallow Water

4.1.1 Dataset

We evaluate on the PDEBench [5] 2D SWE dataset (Radial Dam Break, file `2D_rdb_NA_NA.h5`). The dataset contains $N = 1,000$ independent simulations with varying initial conditions, each producing $h(x, y, t)$ on a 128×128 grid over 101 time steps ($t \in [0, 1]$). We use an 80/20 train-validation split (800/200 scenarios).

4.1.2 Evaluation Metric

Following PDEBench protocol, we report the Relative L2 Error:

$$\varepsilon_{\text{Rel-L2}} = \frac{\|\hat{\eta} - \eta^*\|_2}{\|\eta^*\|_2}, \quad (12)$$

averaged over all validation samples.

4.1.3 Experimental Configurations

Three model configurations are trained on the same dataset and split:

1. **DeepONet – Full-State (Baseline):** Branch receives complete 128×128 initial condition ($d_{\text{in}} = 16,384$).
2. **Partial-DeepONet – Interior Sensors:** Branch receives time-series from 16 sensors on a 4×4 interior grid ($d_{\text{in}} = 1,616$).
3. **Boundary-DeepONet – Edge Sensors:** Branch receives time-series from 16 sensors placed exclusively on the four domain edges ($d_{\text{in}} = 1,616$).

All models use identical architecture and hyperparameters as described in Section 3.3.

4.1.4 Results

Table 1: Relative L2 error on PDEBench 2D SWE (Radial Dam Break). FNO and U-Net results cited from [5].

Model	Branch Input	Rel. L2 (%) ↓	Train Time
<i>External baselines (cited):</i>			
FNO-2D	Full field	5.13	—
U-Net	Full field	12.80	—
<i>Our baseline:</i>			
DeepONet (Full-State)	128×128 IC	3.74	33.9 s
<i>Proposed – partial observation:</i>			
Partial-DeepONet (interior)	4×4 interior $\times T$	3.95	28.4 s
Boundary-DeepONet (edges)	16 edge sensors $\times T$	5.25	27.9 s

4.1.5 Analysis

Baseline validation. Our full-state DeepONet achieves Rel. L2 = 3.74%, outperforming FNO-2D (5.13%) and significantly surpassing U-Net (12.80%) on the same benchmark. This confirms that DeepONet is a competitive foundation, ruling out the possibility that degradation in the partial setting stems from an inherently weak base model.

Partial observation: interior sensors. With 16 sensors distributed across the interior on a 4×4 grid, the Partial-DeepONet achieves 3.95%—nearly matching the full-state baseline (gap of only 0.21 percentage points). This demonstrates that temporal coherence captured by sparse sensors contains sufficient information for accurate full-field reconstruction, despite a $\sim 90\%$ reduction in input dimensionality.

Boundary-only observation. The most practically significant result is the Boundary-DeepONet configuration. All 16 sensors are placed exclusively on the four edges, with zero coverage of the interior. Despite this severe restriction, the model achieves 5.25%, comparable to full-state FNO (5.13%). This demonstrates a clear progression:

- Interior sensors (3.95%): direct interior information yields near-full-state accuracy.
- Boundary-only sensors (5.25%): restricting sensors to edges introduces a genuine inverse problem—reconstructing interior dynamics purely from boundary signals—yet remains competitive with state-of-the-art methods that require complete observations.

4.2 Case Study: Tidal Propagation in the Vinh Bac Bo

4.2.1 Results

4.2.2 Connecting Benchmark and Case Study

Table 2: Comparison between PDEBench benchmark and Gulf of Tonkin case study.

Aspect	PDEBench	Gulf of Tonkin
Domain	Synthetic, flat bottom	Real bathymetry (0–1 500 m)
Grid	128×128 , uniform	110×130 , structured
Branch input	16 sensors $\times$ 101T	10 sensors $\times$ 168T
Rel. L2	3.95% (partial)	21.5%
Complexity	Low (radial dam break)	High (tidal + bathymetry)

5 Discussion

6 Conclusion

References

- [1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhatt, A. Stuart, and A. Anandkumar, “Fourier Neural Operator for Parametric Partial Differential Equations,” in *Proc. ICLR*, 2021.
- [2] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, vol. 3, pp. 218–229, 2021.
- [3] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.
- [4] S. Wang, H. Wang, and P. Perdikaris, “Learning the solution operator of parametric partial differential equations with physics-informed DeepONets,” *Science Advances*, vol. 7, no. 40, 2021.
- [5] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert, “PDEBench: An Extensive Benchmark for Scientific Machine Learning,” in *Proc. NeurIPS*, 2022.
- [6] G. Evensen, “The Ensemble Kalman Filter: theoretical formulation and practical implementation,” *Ocean Dynamics*, vol. 53, pp. 343–367, 2003.

- [7] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” *arXiv:1606.08415*, 2016.
- [8] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. ICLR*, 2019.
- [9] T. T. Nguyen and T. H. Nguyen, “Application of a Godunov type numerical scheme and a domain decomposition technique to the parallel computation of tidal propagation,” *VNU J. Sci., Earth Sci.*, vol. 25, pp. 104–115, 2009.
- [10] Z. Xu et al., “Accelerate Coastal Ocean Circulation Model with AI Surrogate,” *arXiv:2410.14952v2*, 2025.